Deliverable 1

Amalie K. Meltvedt and Maren J. Borgaas

Date: 15.10.2016

GitHub Repository Link

The GitHub repository can be accessed through this link:
https://github.com/amaliemeltvedt/Deliverable_1.git

Importing the necessary libraries

In [1]:
```
# Adding necessary packages
import requests
import pandas as pd
import os
from dotenv import load_dotenv
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
```

Task 1

The client ID is loaded from an environment file to access frost API. Defines the parameters to retrieve air temperature data for 2025 for this task. This code was written using the example code by MET: https://frost.met.no/python_example.html.

In [ ]:
```
# reads the .env file and loads the environment variables including the client I
load_dotenv()
client_id = os.getenv("MET_CLIENT_ID")

# From the task description
# Using P1D to retrieve daily mean temperatures
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'mean(air_temperature P1D)',
'referencetime': '2025-01-01/2025-12-31',
}
```

Sends an HTTP GET request to the frost API with the defined parameters and client ID. Converts the JSON response into python data. Tests whether the request was successfull.

In [53]:
```
# From the MET example code
# Making the HTTP GET request to the Frost API
r = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data and make it into a Python data structure
mean_temperature = r.json()

# From the MET example code
```

```
# Check if the request worked, print out any errors
if r.status_code == 200:
    data = mean_temperature['data']
    print('Data retrieved from frost.met.no!')

else:
    print('Error! Returned status code %s' % r.status_code)
    print('Message: %s' % mean_temperature['error']['message'])
    print('Reason: %s' % mean_temperature['error']['reason'])
```

```
Data retrieved from frost.met.no!
```

The request was successfull and the data was retrieved.

Extracts observation dates and temperatures from JSON into a dataframe. Furthermore, the dates are converted to datetime objects for plotting.

In [54]:
```
# Making a dataframe from the API data, making it easier to analyze
# Filtrating the observations to only include those with a timeOffset of PT0H
df = pd.DataFrame([
    {
        'date': entry["referenceTime"],
        "temperature": observation["value"]
    }
    for entry in data
    for observation in entry["observations"]
    if observation["timeOffset"] == "PT0H"
    ])

# Converts the dates to datetime objects
df["date"] = pd.to_datetime(df["date"])
```

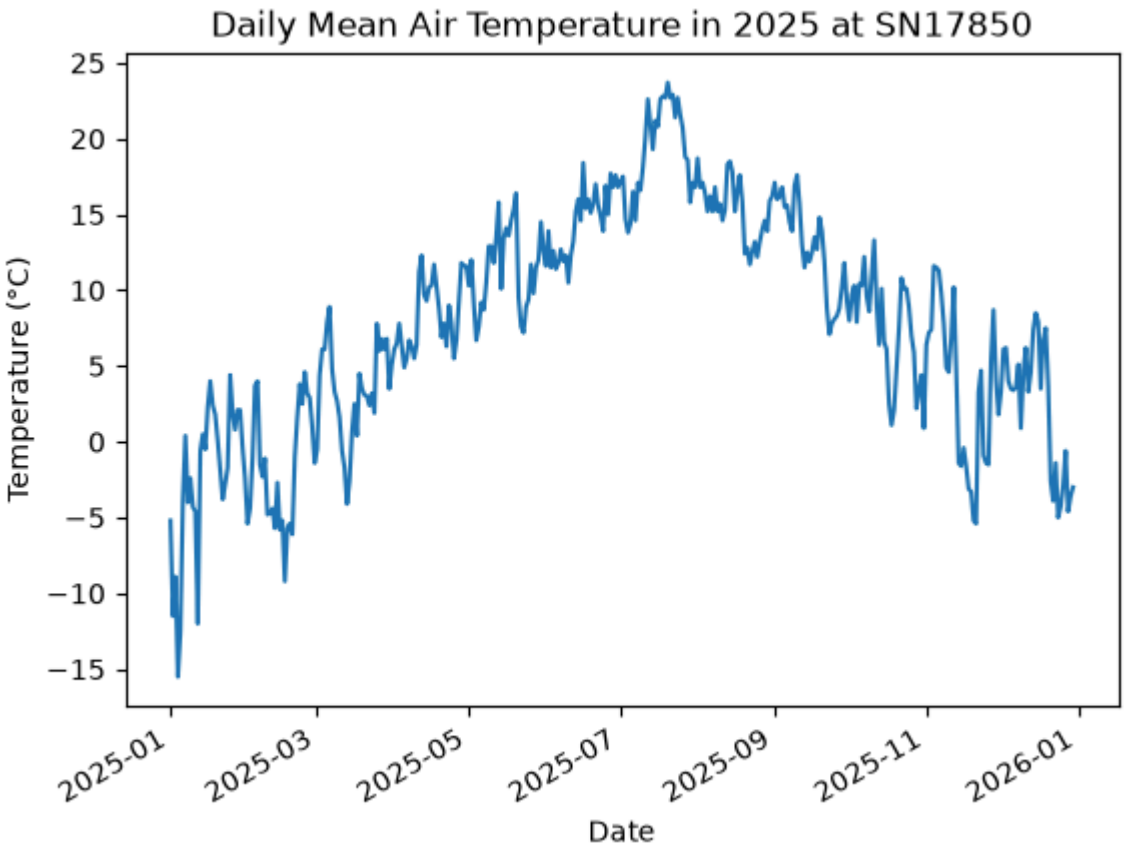
Making a line plot to visualise the daily mean temperatures throughout 2025 in Ås. Labels the axes and formating to improve readability.

In [55]:
```
# Making the plot using dates on the x-axis and temperatures on the y-axis
fig, ax = plt.subplots()
ax.plot(df["date"], df["temperature"])

# Formatting the plot
ax.set_xlabel("Date")
ax.set_ylabel("Temperature (°C)")
ax.set_title("Daily Mean Air Temperature in 2025 at SN17850")

# Make the x-axis labels more readable
fig.autofmt_xdate()

# Show the plot
plt.show()
```



The plot shows that the temperatures varies quite a lot within the months, and it varies between the months as one would expect. The highest temperatures are found in July, and the lowest are found in January.

Calculates the summary statistics for 2025 in a summary table, including mean, median, min and max temperatures.

In [56]:
```
# Making a summary table
summary_table = df[["temperature"]].agg(["mean", "median", "min", "max"]).round(
print("\nSummary Table:")
print(summary_table)
```

```
Summary Table:
        temperature
mean            7.9
median          8.4
min           -15.5
max            23.7
```

The summary table gives that the lowest temperature for 2025 was -15,5 and the highest was 23,7. The mean temperature was 7,9 and the median was 8,4 degrees.

Task 2

To request data for precipitation as well, we reused the MET example code from Task 1 and changed the `elements` parameter to `sum(precipitation_amountP1D)`. The result could then be stored in the variable `precipitation_data`:

In [57]:
```
#  Changing the parameters to find precipitation instead
parameters = {
'sources': 'SN17850',
'elements': 'sum(precipitation_amount P1D)',
'referencetime': '2025-01-01/2025-12-31',
"timeoffsets": "default"
}

# From the example code, modified for precipitation
# Issue an HTTP GET request for precipitation
precipitation_response = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data
precipitation_data = precipitation_response.json()["data"]
```

Dates and values were then extracted and put in a dataframe, then converted to datetime objects in the same way as in Task 1:

In [58]:
```
# Making a dataframe from the API data, making it easier to analyze
# Filtrating the observations to only include those with a timeOffset of PT6H
precipitation_df = pd.DataFrame([
    {
        'date': entry["referenceTime"],
        'precipitation': observation["value"]
    }
    for entry in precipitation_data
    for observation in entry["observations"]
    if observation["timeOffset"] == "PT6H"
])

# Convert dates to datetime objects
precipitation_df["date"] = pd.to_datetime(precipitation_df["date"])
```

The plot was then made in a similar way as in task 1, now with data from both dataframes. `ax2 = ax1.twinx()` makes a new Y-axis on the right side, so that the temperature and precipitation have one axis each.

In [59]:
```
# Making the plot
fig, ax1 = plt.subplots()

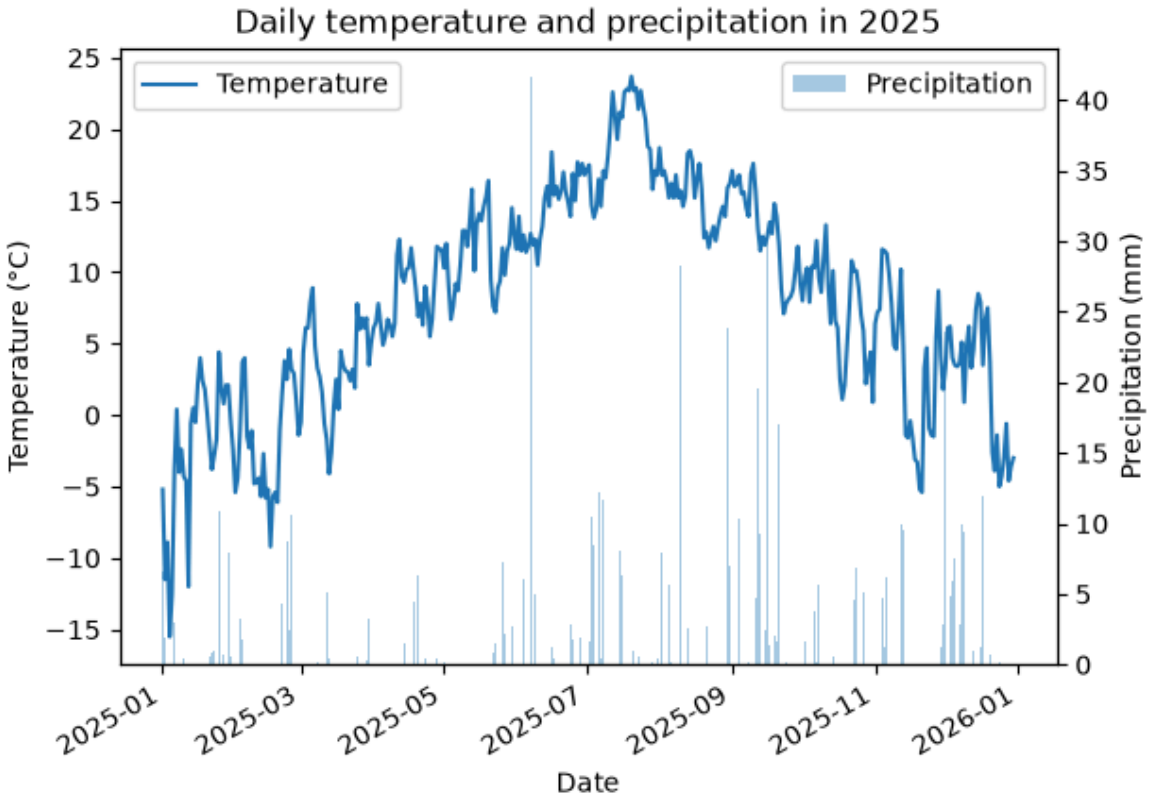
# Temperature
ax1.plot(
    df["date"], df["temperature"],
    label="Temperature"
)
ax1.set_xlabel("Date")
ax1.set_ylabel("Temperature (°C)")

# Precipitation
ax2 = ax1.twinx()
ax2.bar(
    precipitation_df["date"], precipitation_df["precipitation"],
    width=0.5, alpha=0.4, label="Precipitation"
)
ax2.set_ylabel("Precipitation (mm)")

# Adding legends
ax1.legend(loc="upper left")
```

```
ax2.legend(loc="upper right")

# Formatting the plot
plt.title("Daily temperature and precipitation in 2025")
fig.autofmt_xdate()
plt.show()
```



The figure shows a plot illustrating both values. We chose to portray the precipitation amount as bars and the temperatures as a line, to make it easier to differentiate between the two. As expected, the plot shows that the daily temperatures were lower in the winter months, often dipping below zero. Temperature rose as summer approached, and reached a peak around July/August. Precipitation seems to have peaked in June, but overall there appears to have been more days with a higher amount of percipitation during the later months of the year.

Task 3

Retrieves daily mean temperatures from frost API for both 1925 and 2025. Saves the data in a dictionary.

In [60]:
```
# From the task description, reused from task 1
load_dotenv()
client_id = os.getenv("MET_CLIENT_ID")
endpoint = 'https://frost.met.no/observations/v0.jsonld'

# Store data for both years in a dictionary
temperature_data = {}

# Loading mean temperature data from 1925 and 2025
for year in [1925, 2025]:
```

```
    parameters = {
        'sources': 'SN17850',
        'elements': 'mean(air_temperature P1D)',
        'referencetime': f'{year}-01-01/{year}-12-31',
    }

    # Issue an HTTP GET request like previously done
    r = requests.get(endpoint, parameters, auth=(client_id,''))

    json_data = r.json()

    if r.status_code == 200:
        temperature_data[year] = json_data["data"]
        print(f'Data for {year} retrieved from frost.met.no!')
    else:
        print(f'Error retrieving data for {year}! Returned status code {r.status
        print('Message: %s' % json_data['error']['message'])
        print('Reason: %s' % json_data['error']['reason'])
```

```
Data for 1925 retrieved from frost.met.no!
Data for 2025 retrieved from frost.met.no!
```

Extracting dates and temperatures, and storing them in separate dataframes. Converts the dates to datetime objects.

In [61]:
```
# Extract data for 1925 and store in a dataframe
df_1925 = pd.DataFrame([
    {
        "date": entry["referenceTime"],
        "temperature": observation["value"]
    }
    for entry in temperature_data[1925]
    for observation in entry["observations"]
    if observation["timeOffset"] == "PT0H"
])

# Extract data for 2025 and store in a dataframe
df_2025 = pd.DataFrame([
    {
        "date": entry["referenceTime"],
        "temperature": observation["value"]
    }
    for entry in temperature_data[2025]
    for observation in entry["observations"]
    if observation["timeOffset"] == "PT0H"
])

# Convert the dates to datetime objects
df_1925["date"] = pd.to_datetime(df_1925["date"])
df_2025["date"] = pd.to_datetime(df_2025["date"])
```

Creating new plotting dates using the year 2000 (because it is at leap year and therefore includes February 29th), which allows the temperatures to be compared.

In [62]:
```
# Setting the year to 2000 for both datasets
df_1925["plot_date"] = pd.to_datetime(
    "2000-" + df_1925["date"].dt.strftime("%m-%d")
)
df_2025["plot_date"] = pd.to_datetime(
```

```
    "2000-" + df_2025["date"].dt.strftime("%m-%d")
)
```

Making a line chart including the daily mean temperatures for 1925 and 2025, to compare how the temperatures may have changed over the past 100 years.

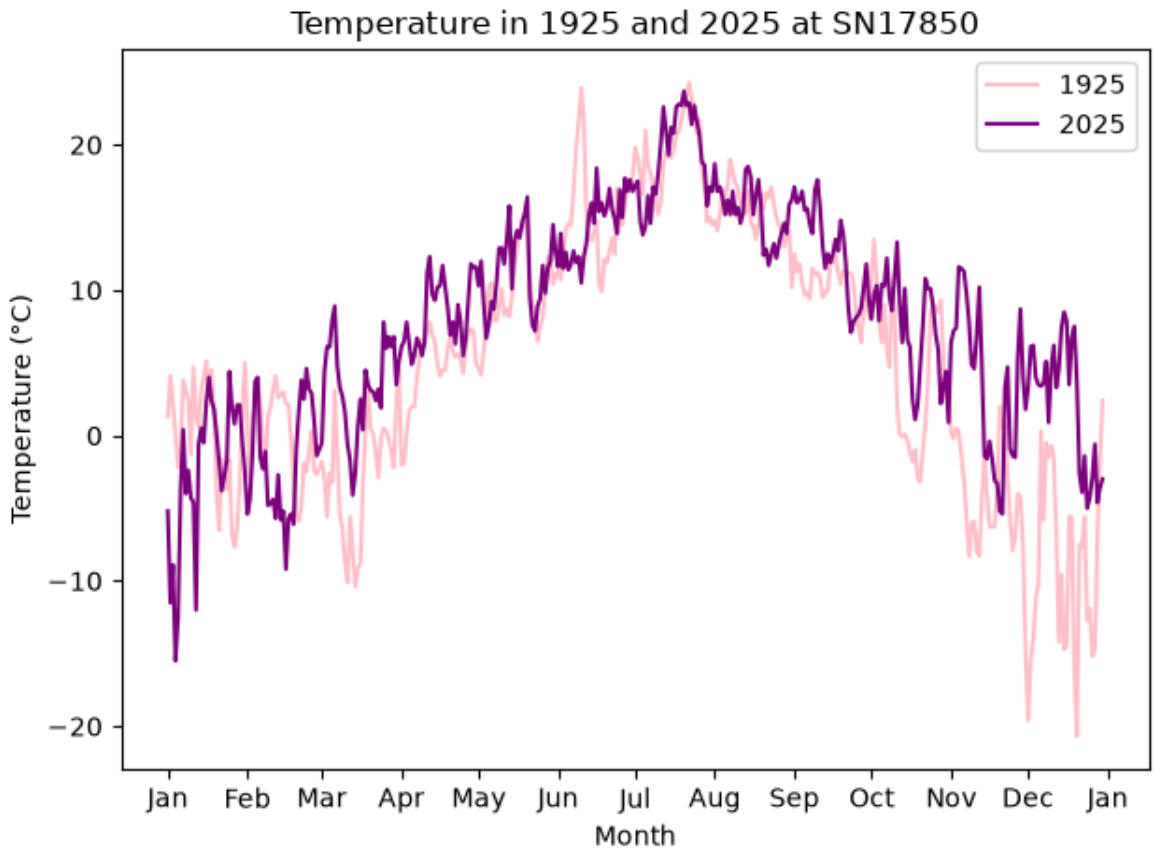
In [63]:
```
# Making the line chart comparing the two years
fig, ax = plt.subplots()

# Plotting the data for both years with different colors and labels
ax.plot(
    df_1925["plot_date"],
    df_1925["temperature"],
    label="1925",
    color="pink"
)
ax.plot(
    df_2025["plot_date"],
    df_2025["temperature"],
    label="2025",
    color="purple"
)

# Labeling the axes and adding a title
ax.set_xlabel("Month")
ax.set_ylabel("Temperature (°C)")
ax.set_title("Temperature in 1925 and 2025 at SN17850")

# Show only month on the x-axis
ax.xaxis.set_major_locator(mdates.MonthLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b"))

# Using ax.legend to see the labels for the two years
ax.legend()
fig.tight_layout()
plt.show()
```



The plot shows how the temperatures changes through the months for the two years. It seems that the maximum temperature for these two years are not very different. However, it seems that the temperature in general is lower throughout the year 1925. In 1925 the temperature reaches about -20 degrees at the lowest, while in 2025 it seems that the lowest temperature was about -15 degrees. Hence this plot gives that the temperatures for the different months have increased over the past 100 years.

Calculating summary statistics for temperatures in 1925 and 2025, including mean, median, min and max temperatures.

In [64]:
```
# Making a summary-table of 1925 and 2025
summary_table = pd.DataFrame({
    "1925": df_1925["temperature"].agg(["mean", "median", "min", "max"]),
    "2025": df_2025["temperature"].agg(["mean", "median", "min", "max"])
}).round(2)

print("\nSummary Table:")
print(summary_table)
```

```
Summary Table:
         1925  2025
mean     5.31   7.9
median   5.30   8.4
min    -20.70 -15.5
max     24.30  23.7
```

Like the plot shows, the overall temperature has increased in the past 100 years. The mean temperature has increased with 2,6 degrees. The median show an even bigger increase of 3,1 degrees. This is a big change, and seems concerning.

Task 4

To look for heatwaves, we first reused the code from Task 1 and switched the `element` parameter to `max(air_temperature P1D)`. The data could then be saved in the variable `temperature_data`.

In [65]:
```
# From the MET example, reused from task 1
# Switched elements to max instead of mean
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
'sources': 'SN17850',
'elements': 'max(air_temperature P1D)',
'referencetime': '2025-01-01/2025-12-31',
}

# From the MET example
# Issue an HTTP GET request for air temp
temperature_response = requests.get(endpoint, parameters, auth=(client_id,''))
# Extract JSON data
temperature_data = temperature_response.json()["data"]
```

In order for the function to work regardless of the station name/id and year, we also fetched the station information and stored it in the variable `station_name`. This means the output should display the right station name based on the station ID provided in the parameters above.

In [66]:
```
# Fetch station information
source_endpoint = 'https://frost.met.no/sources/v0.jsonld'

source_response = requests.get(
    source_endpoint,
    params={'ids': parameters['sources']},
    auth=(client_id, '')
)

source_data = source_response.json()
station_name = source_data["data"][0]["name"]
```

The necessary data was then extracted as before and converted to datetime objects.

In [67]:
```
# Extract dates and values for temperature
temperature_dates = [
    item["referenceTime"] for item in temperature_data
]
temperatures = [
    item["observations"][0]["value"] for item in temperature_data
]

# Convert dates to datetime objects
temperature_dates = pd.to_datetime(temperature_dates)
```

The function `count_heatwaves()` uses a for loop to iterate through the temperatures and dates. It then checks if the temperature surpassed 26 degrees. If it did, the code adds +1 to the `consecutive_days` variable. If the number there is 0, the code will also add

the date to the `start_date` variable, meaning this is the beginning of a heatwave. With each iteration, it adds +1 if the temperature is high enough, and registers it as a heatwave if the number stored in `consecutive_days` reaches 5 or more days. The `end_date` is then added, and both dates are appended into the `heatwaves` list.

In [68]:
```
def count_heatwaves():
    """Finds periods with at least five consecutive days with 27 or above. Retur
    consecutive_days = 0
    heatwaves = []
    start_date = None
    end_date = None

    for date, temperature in zip(temperature_dates, temperatures):
        if temperature >= 27:
            if consecutive_days == 0:
                start_date = date

            consecutive_days += 1
            end_date = date

        else:
            if consecutive_days >= 5:
                dates = f"{start_date.date()} - {end_date.date()}"
                heatwaves.append(dates)

            consecutive_days = 0

    # Checking again in case a heatwave continues after the last day of the data
    if consecutive_days >= 5:
        dates = f"{start_date.date()} - {end_date.date()}"
        heatwaves.append(dates)
    return heatwaves

heatwaves = count_heatwaves()
```

The result is then formatted in the correct format presented in the yml file. This is achieved through another function:

In [69]:
```
# Making a function to print using the yml format
def print_summary(heatwaves):
    """Prints the heatwave dates, weather station ID, station name, and year in
    print("dates:")
    for date in heatwaves:
        print(f"- {date}")

    print(f"station_id: {parameters['sources']}")
    print(f"station_name: {station_name}")
    print(f"year: {parameters['referencetime'][:4]}")

print_summary(heatwaves)
```

```
dates:
- 2025-07-16 - 2025-07-20
station_id: SN17850
station_name: ÅS
year: 2025
```

The output shows a summary of the data collected by the `count_heatwaves()` function. The dates listed show the consecutive period where the degrees surpassed 27. The station ID in this case symbolizes Ås, and the year the data has been collected from is 2025. As shown in the result, there was one heatwave that year, lasting from the 16th of July to the 20th.

Task 5

The function `calculate_moving_avg()` calculates the 7-day moving average for temperature by using the `.mean()` function. `(window=7)` ensures this is done for 7 days at a time.

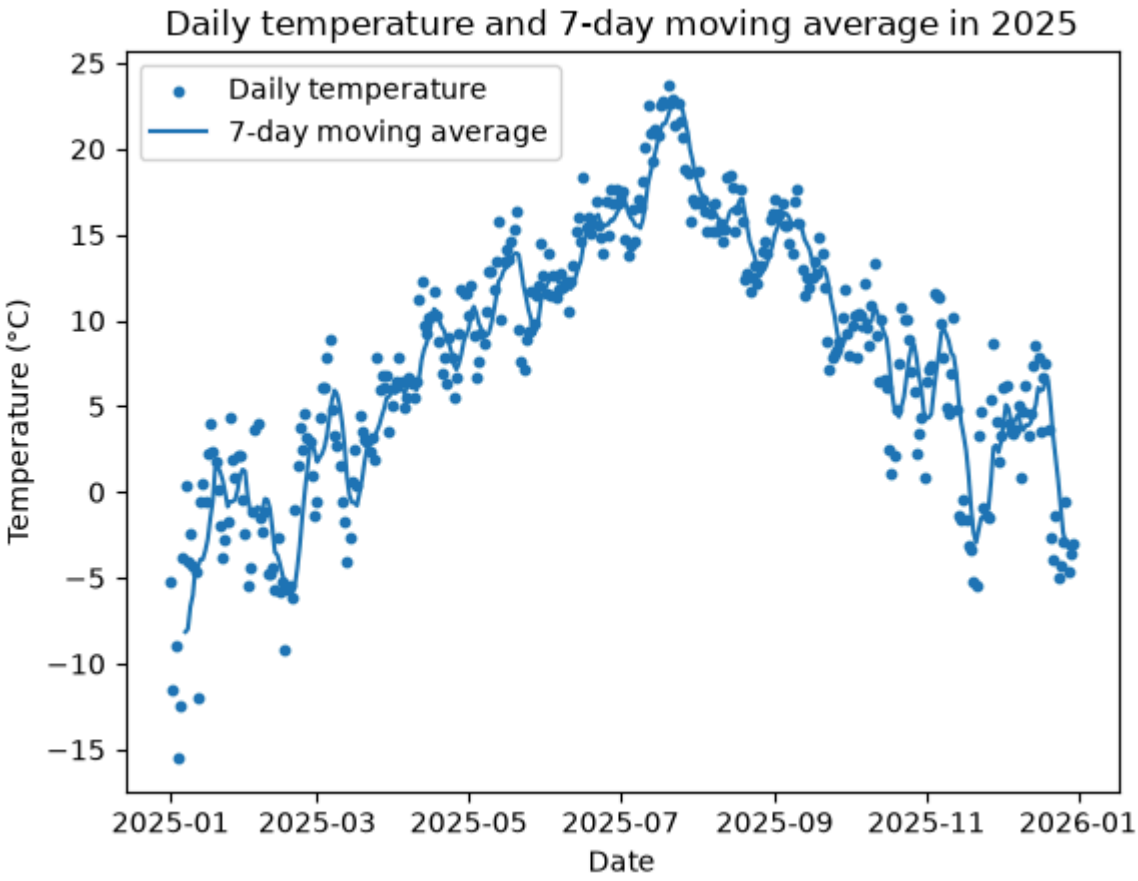
In [70]:
```
def calculate_moving_avg(temperatures):
    """Calculates the 7-day moving average of temperatures. Returns a series con
    moving_avg = temperatures.rolling(window=7).mean() # calculates the mean of
    return moving_avg
```

The function can be used on the 2025 mean temperature from Task 1 by using the dataframe created in Task 1. The plot is then plotted showing the moving average as a line and the daily temperature as points.

In [71]:
```
df["moving_avg"] = calculate_moving_avg(df["temperature"])

plt.scatter(df["date"], df["temperature"], label="Daily temperature", s=10)
plt.plot(df["date"], df["moving_avg"], label="7-day moving average")

plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Daily temperature and 7-day moving average in 2025")
plt.legend()
plt.show()
```



The figure shows the average daily temperatures as points along the Y-axis, and the 7-day moving average as a line along the X-axis. The line seems to follow the daily temperature points as expected. The figure tells us that the highest temperature was around July/August, and this occured during a week where the 7-day moving average was also at its highest. As expected, temperatures were lower during the winter months and peaked in the middle of summer. At the beginning of the year in January and February, there are a few outliers from the 7-day moving average where the daily temperatures were much lower than the mean.